C++ COMPLETE REFERENCE LIBRARY • MODULE 18

18 TEMPLATES — Complete Theory & Study Book

Comprehensive Offline Theory, Memory Diagrams, Classroom C++ Programs, Practice Questions & Solutions

Module Table of Contents

1. **Module Overview & Study Guide** (README)
2. **Theory Chapter:** 01 function templates and instantiation
3. **Theory Chapter:** 02 class templates and specialization
4. **Theory Chapter:** 03 modern cpp concepts overview
5. **Visual Memory & Concept Diagrams** (1 Vector Diagram)
6. **Annotated C++ Source Programs** (3 Programs)
7. **Practice Question Bank:** conceptual questions
8. **Practice Question Bank:** debugging questions
9. **Practice Question Bank:** predict the output
10. **Practice Question Bank:** programming questions
11. **Step-by-Step Solutions:** conceptual solutions
12. **Step-by-Step Solutions:** output solutions
13. **Step-by-Step Solutions:** programming solutions

18 — Templates (Generic Programming)

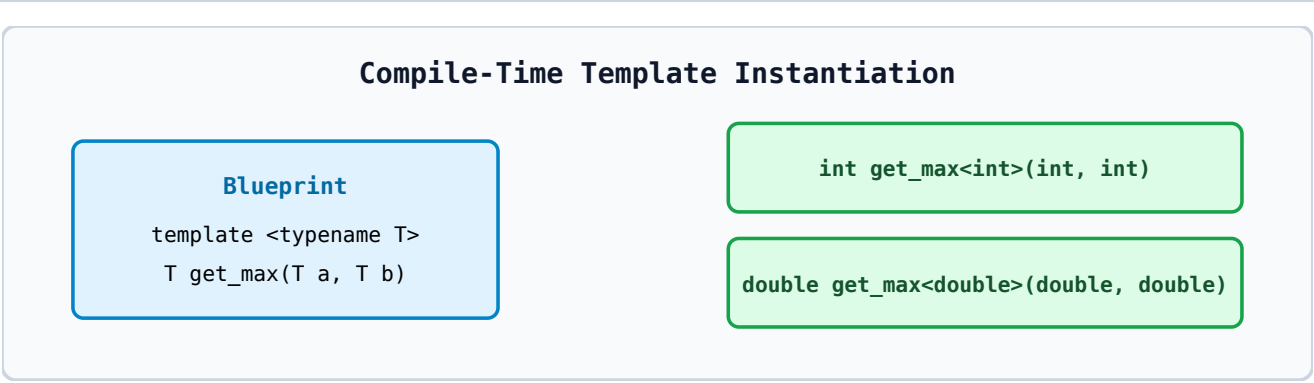
This module covers Function Templates (`template <typename T>`), Class Templates (`box<T>`), multi-parameter templates (`template <typename T, typename U>`), template specialization, and a C++20/C++23 introduction to `concepts`.

Directory Contents

- **Theory (`01_THEORY/`)**:
  - `01_function_templates_and_instantiation.md` (./01_THEORY/01_function_templates_and_instantiation.md)
  - `02_class_templates_and_specialization.md` (./01_THEORY/02_class_templates_and_specialization.md)
  - `03_modern_cpp_concepts_overview.md` (./01_THEORY/03_modern_cpp_concepts_overview.md)
- **Programs (`02_PROGRAMS/`)**:
  - `01_function_template_max_and_swap.cpp` (./02_PROGRAMS/01_function_template_max_and_swap.cpp) (Category A)
  - `02_class_template_pair_box.cpp` (./02_PROGRAMS/02_class_template_pair_box.cpp) (Category A)

  - `03_template_deduction_compile_error.cpp` (./02_PROGRAMS/03_template_deduction_compile_error.cpp) (Category B)
- **Practice Questions (`03_PRACTICE_QUESTIONS/`)** & **Detailed Solutions (`04_DETAILED_SOLUTIONS/`)**
- **Diagrams (`05_DIAGRAMS/`)**: `diagram_01_template_instantiation.svg` (./05_DIAGRAMS/diagram_01_template_instantiation.svg)

01 — Function Templates and Compile-Time Instantiation



Template Instantiation

CPP

```cpp
template <typename T>
T find_max(T a, T b) {
    return (a > b) ? a : b;
}
```

- `typename T` and `class T` inside `template <...>` are **100% interchangeable**.
- **Template Instantiation**: A template itself generates no machine code until called with concrete types (`find_max(10, 20)` instantiates `int find_max<int>(int, int)` at compile time).
- **Strict Type Deduction**: `find_max(10, 3.5)` fails to deduce `T` because the first argument is `int` and the second is `double`! Either pass matching types or specify explicitly: `find_max<double>(10, 3.5)`.

02 — Class Templates and Template Specialization

CPP

```cpp
template <typename T>
class storage_box {
private:
    T value;
public:
    storage_box(T v) : value(v) {}
    void display() const { cout << value << endl; }
};
```

In `main()`, you create instances such as `storage_box<int> b1(42);` and `storage_box<double> b2(3.14);`.

03 — Modern C++20 / C++23 `concepts` Overview

In C++20 and C++23 (`#include <concepts>`), you can constrain a template parameter using `requires` so the compiler produces a clear error message if a non-numeric type is passed:

CPP

```cpp
template <typename T>
requires (sizeof(T) ⩾ 4)
T add_values(T a, T b) { return a + b; }
```

Visual Concept & Memory Diagrams (18 TEMPLATES)

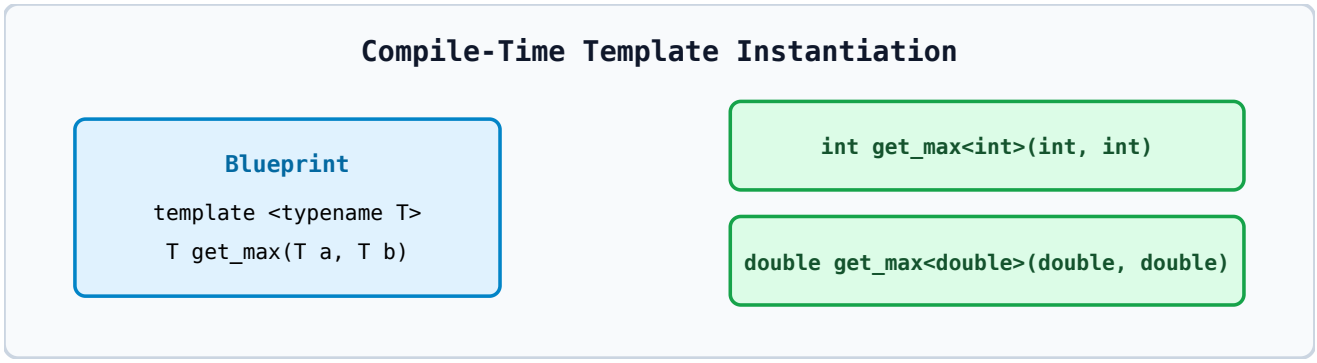


Diagram: diagram 01 template instantiation

Annotated C++ Programs (18 TEMPLATES)

Program: `01_function_template_max_and_swap.cpp`

C++ SOURCE FILE • 01_function_template_max_and_swap.cpp

```cpp
// Program Title: Generic Swap and Maximum Using Function Templates
// Topic: Templates — Function Templates
// Purpose: Demonstrate a single template function working for both int and double.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/01_function_templates_and_instantiation.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

template <typename T>
T get_max(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    cout << "Max int: " << get_max(15, 25) << endl;
    cout << "Max double: " << get_max(4.5, 2.1) << endl;
    return 0;
}

/*
Expected Deterministic Output:
Max int: 25
Max double: 4.5
*/
```

Program: `02_class_template_pair_box.cpp`

C++ SOURCE FILE • 02_class_template_pair_box.cpp

```cpp
// Program Title: Classroom Class Template With Two Type Parameters
// Topic: Templates — Class Templates
// Purpose: Demonstrate a generic key-value pair class template.
// Target C++ Standard: C++23 (compatible with C++17/C++20)
// Related Theory File: ../01_THEORY/02_class_templates_and_specialization.md
// Program Classification: Category A — Valid and well-defined program

#include <iostream>
using namespace std;

template <typename T1, typename T2>
class key_value {
private:
    T1 key;
    T2 val;

public:
    key_value(T1 k, T2 v) : key(k), val(v) {}

    void display() const {
        cout << "Key: " << key << ", Value: " << val << endl;
    }
};

int main() {
    key_value<int, double> kv(101, 98.5);
    kv.display();
    return 0;
}

/*
Expected Deterministic Output:
Key: 101, Value: 98.5
*/
```

Program: `03_template_deduction_compile_error.cpp`

C++ SOURCE FILE • 03_template_deduction_compile_error.cpp

```cpp
// INTENTIONAL COMPILE-TIME ERROR DEMONSTRATION
// Program Title: Conflicting Template Argument Deduction (T vs T)
// Topic: Templates — Template Argument Deduction Rules
// Purpose: Show why get_max(10, 3.14) fails to deduce a single type T when parameters are
(T, T).
// Target C++ Standard: C++23
// Related Theory File: ../01_THEORY/01_function_templates_and_instantiation.md
// Program Classification: Category B — Intentional compilation-error demonstration

#include <iostream>
using namespace std;

template <typename T>
T get_max(T a, T b) {
    return (a > b) ? a : b;
}

int main() {
    // COMPILE ERROR: deduced conflicting types for parameter 'T' ('int' and 'double')
    cout << get_max(10, 3.14) << endl;
    return 0;
}

/*
Why Compilation Fails:
Template argument deduction does not perform implicit arithmetic promotions on template
parameters 'T'.
Fix: Call 'get_max<double>(10, 3.14)' or use two template parameters 'template <typename T1,
typename T2>'.
*/
```

Conceptual Questions — Module 18: Templates

Q1. Is there any difference between `template <class T>` and `template <typename T>`? **Q2.** Explain how Function Templates differ from Preprocessor Macros (`#define MAX(a,b)`).

Debugging Questions — Module 18

Q4. How do you fix `get_max(5, 7.2)` when `template <typename T> T get_max(T a, T b)` fails to compile?

Predict the Output Questions — Module 18

Q3. Does `template <typename T> void count_calls()` share a single `static int count` across `count_calls<int>()` and `count_calls<double>()`, or does each instantiation get its own separate `static int count`?

Programming Questions — Module 18

Q5. Write a C++23 classroom program implementing a generic `simple_stack<T>` class template with `push(T)` and `pop()`.

Detailed Solutions — Conceptual Questions (Module 18)

- **Q1 Solution**: No difference in a template parameter declaration; both `class` and `typename` accept both fundamental types (`int`) and class types.
- **Q2 Solution**: Templates are type-safe, obey C++ scope rules, and evaluate arguments exactly once, whereas `#define MAX(a,b) ((a)>(b)?(a):(b))` performs blind text substitution and evaluates `a++` twice!

Detailed Solutions — Predict the Output & Debugging (Module 18)

- **Q3 Solution**: Each distinct template instantiation (`count_calls<int>` vs `count_calls<double>`) is a separate function with its own independent `static int count` variable!
- **Q4 Solution**: Specify the template argument explicitly: `get_max<double>(5, 7.2)`.

Detailed Solutions — Programming Questions (Module 18)

CPP

```cpp
#include <iostream>
using namespace std;

template <typename T>
class simple_stack {
    T data[10];
    int top_idx = -1;
public:
    void push(T val) { data[++top_idx] = val; }
    T pop() { return data[top_idx--]; }
};

int main() {
    simple_stack<int> st;
    st.push(10);
    st.push(20);
    cout << st.pop() << " " << st.pop() << endl;
    return 0;
}
// Expected Output: 20 10
```